

Signals and Systems 2025/2026 (LEEC & LEBIOM)

Lab Guide

November 2025

Introduction

This document describes the laboratory component of the Signals and Systems course. Its goal is to provide students with practical experience and a deeper understanding of the theoretical concepts through experimentation and visualisation. Working with both synthetic and real-world signals, you will gain insights into the behavior of systems in time and frequency, develop a hands-on grasp of the effects of operations like convolution, filtering, or Fourier transformation, and hone your problem-solving skills. When answering the questions in this guide and commenting on the results, you are expected to exercise critical thinking; rather than simply describing what was obtained in each experiment, it is more important to interpret the results and discuss them (Were they expected? Do they make sense? What do they mean?)

Students shall work in groups of two or three and submit via Fénix, by the end of the course, a concise report of the lab work containing the answers to selected questions in this guide, supporting code and results, and any additional comments deemed appropriate (note that questions in this guide may span several paragraphs and end with a horizontal line when followed by regular text). See the lab section of the course web page for the list of questions to be prepared for each lab session and possibly discussed in class, and for those that should be included in the report. The guide includes other activities and questions, not marked with Q, which you are strongly encouraged to address but should not cover in your report. The report consists of a mixed notebook with executable code and text (Matlab live script, Jupyter notebook, Colab notebook...), as well as a pdf generated from it. The pdf should not exceed the equivalent of 10 pages. Groups formed only by Portuguese-speaking students should write the report in Portuguese, others in English. Grading will be individual, based on the submitted materials and on your performance in class.

This guide provides example code in Matlab. Supporting files are provided in Matlab and Python, and you may use either programming language in your report. If using Python (with SciPy), however, it is up to you to adapt the code. Throughout the work, students will work mainly with discrete-time signals using signal-processing software. However, several of the concepts and tools pertain to continuous-time signals, and we will handle those rather informally until the section on sampling. For example, we will generate sampled versions of continuous-time signals using a discretized time axis, but when looking at their spectra we will tweak things to make it look as though these are “native” continuous-time signals and spectra (which is *not* the case), hiding their true discrete-time nature. Or we might play these signals on the speakers and listen to the output without worrying much about the underlying processing needed to generate true continuous-time acoustic signals from a discretized computer-based representation.

1 Signals

Sinusoidal signals

In this section you will generate sinusoidal signals and listen to them. We will define the granularity of the discretized time axis (i.e., how many "time ticks" it contains per second) with the variable `samplerate`.

Start by generating a 2 s time axis as

```
samplerate = 16e3;  
t = (0:1/samplerate:2)';
```

Then, generate variable `x` as a sinusoid with 15 Hz frequency

```
x = cos(2*pi*15*t);
```

Visualise the sinusoid with

```
plot(t,x); grid on
```

Locate the figure window, which may be hidden behind other windows. Become familiar with the controls to pan, zoom, turn data tips on and off, etc. How many periods of the sinusoid should you see per second? Note that the next plot command that you issue will overwrite the current plot unless you use `hold on` or create a new figure window with the `figure` command.

Now change the time axis duration to 1 s and the frequency of the sinusoid to 1000 Hz. Recreate the signal in `x` and visualise it. Initially, the plot will look like a solid rectangle, and you will have to zoom in to see its rapid oscillations.

Listen to the sinusoidal signal using the command

```
soundsc(x,samplerate)
```

Caution! When using earphones make sure that the volume is not too loud before inserting them in your ears.

Test your hearing limits

For the next test, set

```
samplerate = 48e3;
```

Generate several sinusoids with different frequencies and check the limits of your hearing. For reasons that will become clear later on, do not test frequencies above `samplerate/2`, i.e., 24 kHz. For this type of test it may be more convenient to generate a sinusoidal-like signal with linearly increasing or decreasing frequency (this is often known as a *linear chirp*) and measure how much time elapses before you stop hearing the sound. Knowing the start and stop frequencies, the total duration of the signal, and the elapsed time when you started/stopped hearing, it is a simple matter to derive the corresponding instantaneous frequency and thus establish the upper and lower limits of your hearing (regular-quality headphones and computer sound cards will influence the results, so don't take them too literally).

For example, to generate a linear chirp with total duration 5 s that starts at 0 Hz and attains a final frequency of 20 kHz use

```
t = (0:1/samplerate:5)';  
x = chirp(t,0,5,20e3);
```

Listen to the signal and visualise it. Try Matlab's Signal Analyzer App to interactively view the signal in time and frequency. To test your lower hearing limit it may be a good idea to use a decreasing linear chirp. To proceed to other topics, set `samplerate` back to its original value

```
samplerate = 16e3;
```

Periodic signals

Generate, over an interval of 2 s, the sum of sinusoids $x(t) = \cos(2\pi f_1 t) + \cos(2\pi f_2 t)$ with $f_1 = 30$ Hz and $f_2 = 35$ Hz, which is a periodic signal.

Q1.1 Determine the period of $x(t)$ theoretically and check that it agrees with the period measured directly in the plot of `x`.

The function `ginput` is very useful for this, as it allows you to interactively click points on a plot using a hairline cursor and then obtain their horizontal and vertical coordinates as workspace variables. Typically, simply invoke it as

```
[xi,yi] = ginput;
```

to get the horizontal and vertical coordinates in vectors `xi` and `yi`, respectively, then click away and hit `<enter>` when you're done marking points

Q1.2 As discussed above, this "continuous-time" signal `x` in Matlab is actually a discrete-time signal. Determine its theoretical period in samples and check against the direct measurement taken from `plot(x)`.

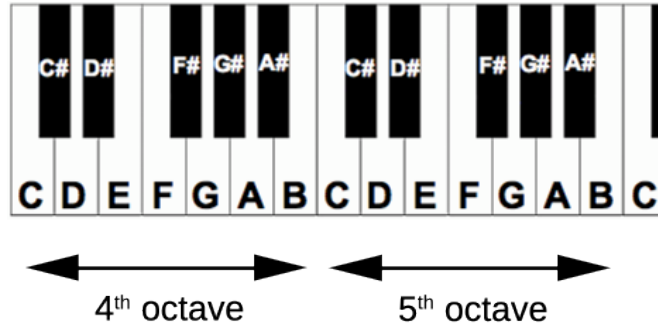
If you now downsample it by a factor of 7

```
y = downsample(x,7);
```

i.e., keeping only one sample for every 7 and discarding the other 6, what will be the period of `y`, in samples? Why does this only seemingly disagree with direct measurement from `plot(y)`?

Musical notes

Each musical note corresponds to a well-defined frequency. The following figure shows two octaves of a piano keyboard, the 4th being the "middle C" octave that is a reference for musicians.



Keep in mind the following facts, which allow you to compute the absolute frequency of any musical note:

- The frequency of A4 is 440 Hz
- The musical scale is logarithmic. A full octave always corresponds to a 2:1 ratio. E.g., the frequency of C5 is double that of C4
- The ratio of frequencies between consecutive notes, including *sharp* ones labeled as # in the figure, is constant. Since there are 12 such notes in an octave, that ratio has to be $\sqrt[12]{2}$ to preserve the 2:1 ratio between octaves
- The ordering of notes in the "middle C" octave by ascending frequency is C4, C#4, D4, D#4, E4, F4, F#4, G4, G#4, A4, A#4, B4, and similarly for any other octave (in Portuguese notation, Dó4, Dó#4, Ré4, Ré#4, Mi4, Fá4, Fá#4, Sol4, Sol#4, Lá, Lá#4, Si4)

Q1.3 Determine the frequencies of B2, C3, D3, E3, G3.

Use the `seqsin` function to generate a sequence of sinusoids and silence gaps. For example, the following

```
x = seqsin(samplerate, 440, 0.3, 0, 0.5, 466.164, 0.15);
```

will assign to `x` the samples of a sinusoid with frequency 440 Hz lasting for 0.3 s, followed by a silent gap (or *rest*) of 0.5 s and a sinusoid of frequency 466.164 Hz lasting for 0.15 s. In shorthand, we will represent this as "A4(0.3), R(0.5), A4#(0.15)".

Now create `x` as the following sequence and listen to the resulting signal:

```
R(0.7), E3(0.4), R(0.3), E3(0.3), G3(0.4), E3(0.4), D3(0.3), C3(0.8), R(0.2), B2(0.8),
```

```
R(0.2), E3(0.4), R(0.3), E3(0.3), G3(0.4), E3(0.4), D3(0.3), C3(0.4), D3(0.4), C3(0.3) B2(0.3), R(0.7)
```

You just accomplished a very basic technique for assembling musical signals. Most electronic instruments use more elaborate ways of generating sounds, but at their core there is usually something akin to the above.

Notes:

The sound of a sinusoid has a relatively "poor" timbre. You can obtain something a bit more interesting with `sound(1.5*x)`, as this scales the samples to numeric values outside the range $[-1, 1]$, but `sound` will clip (saturate) those values back to $[-1, 1]$ before playing. This means that what you hear will effectively be generated by a non-sinusoidal signal, whose timbre is more complex than that of a sinusoid. We will look at periodic non-sinusoidal signals in a later section of this guide.

You may use `plot` or Matlab's Signal Analyzer App to look at the music signal. Notice how each sinusoid starts/ends smoothly when transitioning to silence or to another sinusoid. If transitions were abrupt you would hear annoying "clicks" between notes. Consecutive notes with the same frequency would also sound as a single note with longer duration, rather than two distinct notes.

The initial and final silence gaps are inserted to isolate the "popping" sound made by some computer sound cards when switching on or off from the actual melody.

You can create a polyphonic signal by generating signals for individual voices as outlined above, and then playing their weighted sum using the `sound` command.

Unit impulse and unit step

You may define the unit step, or Heaviside function, in a very simple way as an inline function in Matlab

```
u = @(t) (t >= 0);
```

Generate a time variable spanning an interval of 4 s. Use `plot(t,u(t-t0))` to visualize the (shifted) rectangle for different choices of the offset t_0 .

Q1.4 Function `u` can handle any affine transformation of the time variable. Yet, $u(at + b)$ with arbitrary a and b can simply be written as a time shift and time reflection $u(\pm t - t_0)$ with suitable t_0 . Justify this and find the appropriate values of factor ± 1 and t_0 given a and b . Check your results experimentally for different choices of a and b .

The unit impulse $\delta(t)$ is formally the derivative of the step function $u(t)$, which is infinite at $t = 0$ and zero elsewhere. Because directly working with such a function in practice is problematic, it is common to define a differentiable approximation to the step function $u_\Delta(t)$ and the approximate impulse $\delta_\Delta(t)$ as its derivative.

```
s = @(t) (t >= 0).*t;           % Unit ramp
uD = @(t,D) (1/D)*(s(t+D/2)-s(t-D/2)); % Approximate unit step
duD = @(t,D,h) (uD(t+h/2,D)-uD(t-h/2,D))/h; % Numeric derivative
deltaD = @(t,D) duD(t,D,1e-6); % Approximate unit impulse
```

The parameter Δ defines the width of the region around $t = 0$ where u_Δ transitions from 0 to 1, and where δ_Δ has nonzero support. Generate a time variable spanning a symmetric interval around zero and check `plot(t,deltaD(t-t0,D))` for different choices of D and t_0 .

Visualise the approximate impulse `deltaD(a*t,D)` for different values of the scaling parameter a . Explain why these results are consistent with the following known property of the ideal impulse

$$\delta(at) = \frac{1}{|a|} \delta(t)$$

2 Systems

Consider the continuous-time system whose input-output mapping is defined by

$$y(t) = x(t) + 0.5x(t - 0.6)$$

where x and y denote the input and output, respectively. This is implemented by the following function

```
system1 = @(x) filter([1 zeros(1,round(0.6*samplerate) - 1) 0.5],1,x);
```

Obtain the output of the system for $x(t) = u(t) - u(t - 2)$ with a symmetric time variable spanning 8 s as

```
t = (-4:1/samplerate:4)';
x = u(t) - u(t-2);
y = system1(x);
plot(t,y); grid on
```

Check that the input and output are as expected. Similarly, check the response of this system to the approximate unit impulse $\delta_{\Delta}(t)$.

Q2.1 Read the following clip of a speech signal

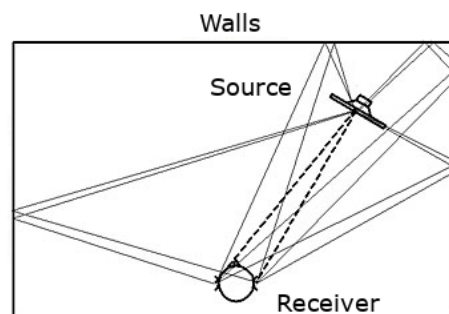
```
[x,samplerate] = audioread('echoin.wav');
```

Apply it to `system1` and listen to the output. Explain what you heard.

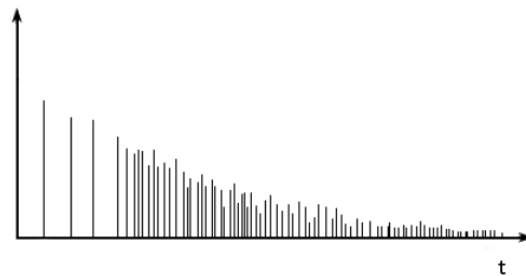
Application: Acoustic fingerprinting

For linear and time-invariant (LTI) systems, the impulse response fully characterises the input-output mapping. Specifically, the output of the system to any input is obtained by convolving it with the impulse response. This section illustrates the process with a simple application where a room imparts an "acoustic fingerprint" on voice signals that propagate between a source and a receiver.

Consider the figure that shows a room inside which an acoustic source and a receiver are positioned. Sound propagates over multiple reflected paths on the walls, floor and ceiling, creating a pattern of echos whose attenuations and delays increase as the path length and number of reflections increase.



This translates into an impulse response between the source and receiver that is formed by a set of scaled impulses, each having a particular delay and magnitude that is associated with a specific propagation path. The following figure illustrates such an impulse response for a strongly reverberating room.



Q2.2 Explain the general features of the impulse response shown above. Why are there fewer and stronger impulses for small delay? Why do amplitudes show a decreasing trend as the delay increases?

The supplied function `rir.m` computes the impulse response of a room given the coordinates of a source and receiver. Start by generating and visualizing a sample room impulse response as follows:

```
mic = [1 2 1];
n = 12;
r = 0.7;
rm = [15 15 5];
src = [5 5 2];
h = rir(samplerate, mic, n, r, rm, src);
```

Vectors `rm`, `src` and `mic` define the size of the room and the 3D positions of the source and receiver, respectively. Variable `r` defines the reflectivity of the walls. Repeat for different positions of the source, receiver, and reflectivity, and see how the impulse response changes.

Load the previously used voice clip, take it as the source signal, create the received signal, and listen to it

```
[x,samplerate] = audioread('echoin.wav');
y = conv(h,x);
soundsc(y,samplerate)
```

Repeat the test for various positions and reflectivities and check if you can perceive any differences.

Q2.3 Now load another sound clip containing the input signal `x` and the output `y` after propagation over a room simulated by `rir`

```
load fingerprint.mat
```

The receiver is located in the geometric center of the room, whose walls are made of concrete (`r = 0.9`). The door is located at `[0 0 *]`, and the source is known to be located close to it, at a height of 2 m, but its exact x/y position is uncertain. The room dimensions could be `rm = [5 4 4]` (smaller

room) or $\mathbf{rm} = [20 \ 15 \ 10]$ (large hall). Start by playing $\mathbf{x}$ and $\mathbf{y}$. Based on your own simulations of acoustic propagation in this room, driven by $\mathbf{x}$, and taking the parameter $\mathbf{n} = 12$ as before, which of the two possible room geometries is more likely?

Q2.4 Now generate a sinusoidal signal $x_1(t)$ according to the the specifications given in the lab class and calculate the output $y_1(t)$ from **system1**. Using the decomposition of a sinusoid as a sum of complex exponentials, show that $y_1(t)$ is also a sinusoid of the form $y_1(t) = A \cos(\omega t + \phi)$ and determine the values of $A \in [0, +\infty[$, $\omega \in \mathbb{R}$, and $\phi \in]-\pi, \pi]$. Keep in mind that the arguments of standard trigonometric functions are given in **radians**.

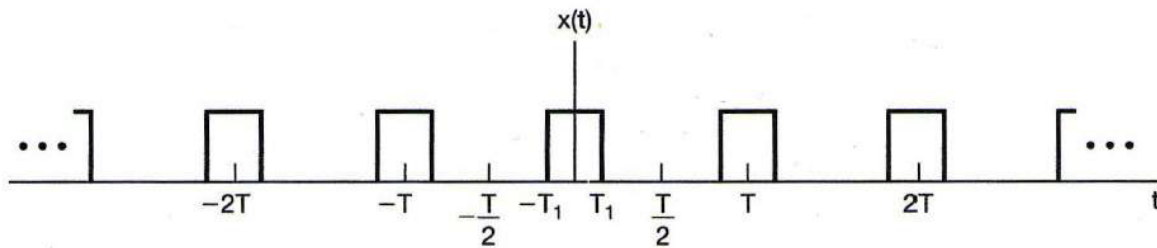
Invoke **system1** to obtain the numeric response to $\mathbf{x1}$. Plot it jointly with the sinusoid whose parameters you determined above and check that they match. Keep in mind that, because you are working with a limited interval of the (infinite) time axis, the generated input is effectively taken as zero outside that interval. At the edges of the time axis you will see transient effects in the output of **system1** due to the temporal misalignment of the input signal and its delayed and scaled replica internally produced by **system1**.

Q2.5 Now you should characterize a "mystery" *discrete-time* system **system2** whose internal details are not available to you. Viewing it as a black box, you should design (multiple) inputs $\mathbf{x}$, defined over suitable discrete-time spans $\mathbf{n}$ (akin to $\mathbf{t}$ above for continuous-time signals and systems), observe the outputs $\mathbf{y} = \mathbf{system2}(\mathbf{x}, \mathbf{n})$, and from those classify the system according to linearity, time invariance, memory, causality, stability, and invertibility. The function assumes that the input signal is zero outside the given discrete-time range.

Describe the experiments, the input signals, the results, and your conclusions. For each property, were you able to reach a definitive yes/no conclusion? Document only the experiments that were relevant to draw conclusions.

3 Fourier Series

Consider the periodic signal (partially) represented in the figure



This signal may be represented as the Fourier series

$$\tilde{x}(t) = \sum_{k=-\infty}^{+\infty} a_k e^{jk\omega_0 t}$$

where

$$a_k = 2 \frac{T_1}{T} \frac{\sin(k\omega_0 T_1)}{k\omega_0 T_1}, \quad \omega_0 = \frac{2\pi}{T}$$

and $T_1 = T/6$.

We will consider approximations to $\tilde{x}$ given by the truncated series

$$\tilde{x}_N(t) = \sum_{k=-N}^N a_k e^{jk\omega_0 t}$$

for different values of N .

Using ω_0 and a_k obtained previously build $\tilde{x}_N(t)$ in a symmetric time interval spanning 10 s for, at least, $N = 1, 2, 3, 4$, and 5. Since $\tilde{x}(t)$ is real, you may alternatively express $\tilde{x}_N(t)$ as a sum of sinusoidal terms with real coefficients

$$\tilde{x}_N(t) = A_0 + \sum_{k=1}^N A_k \cos(k\omega_0 t + \varphi_k)$$

where $A_0 = a_0$ and, for $k \geq 1$, $A_k = 2|a_k|$, $\varphi_k = \arg a_k$.

Q3.1 Create a superimposed plot of all the $\tilde{x}_N(t)$. Zoom in and observe the details of the waveforms. Do they behave as expected? Comment on the results. The following code sample demonstrates how the approximation could be compactly computed for a 80 Hz unipolar square wave with 33% duty cycle. Notice the modified a_k (how?) such that the waveform switches at time $t = 0$ instead of $t = -T_1$ shown in the figure

```
t = (-320:320)'/samplerate;
w0 = 2*pi*80;
T = 2*pi/w0;
T1 = T/6;
ak = @(k) sin(k*w0*T1)./(k*pi).*exp(-1i*k*w0*T1);
a0 = 2*T1/T; % Segregate a0, which often has a different expression

N = 100; % Order of the Fourier expansion
xN = exp(1i*w0*t*(-N:N))*[ak(-N:-1) a0 ak(1:N)].';
plot(t,real(xN)); grid
```

Now visualize the spectra of these signals using the Fourier transform. You may invoke `freqz`, which computes the Fourier Transform in discrete time, and scale the magnitude by `1/samplerate` to match the continuous-time Fourier transform. A wrapper function `cftransform` is provided for convenience

```
[XN,f] = cftransform(xN,find(t == 0),1e4,samplerate);
plot(f,abs(XN))
```

Note that, in general, $\mathbf{X}$ will be complex, hence the code above only plots its magnitude using `abs`. You may also want to examine `angle`, `real`, or `imag`. There are more efficient ways of obtaining a Fourier transform than using `freqz`, at the cost of some additional technicalities that are not warranted here.

Q3.2 Visualize the spectra of the signals $\tilde{x}_N(t)$ created above, zooming in as appropriate. Do these have the structure that you expected?

Consider now the periodic signal provided to you in the lab session, which can once again be represented by the Fourier Series

$$\tilde{x}(t) = \sum_{k=-\infty}^{+\infty} a_k e^{jk\omega_0 t}$$

Q3.3 Find ω_0 and the general expression for the Fourier coefficients a_k for this new signal and also provide the numeric values for a_0 , a_{-1} and a_3 . Use known signal-transform pairs and properties of the Fourier series to obtain your answer, *not* direct evaluation of the Fourier integrals. As before, create a superimposed plot of truncated series $\tilde{x}_N(t)$ with increasing N over a time span of at least 5 periods, as a means to check your computation of the Fourier coefficients.

4 Frequency response

The frequency response of an LTI system, $H(j\omega)$, may be interpreted as the (complex-valued) gain that the system provides for a complex exponential $e^{j\omega t}$ at its input, yielding an output $H(j\omega)e^{j\omega t}$.

Conceptually, one could measure $H(j\omega)$ for a set of frequencies $\omega_1, \omega_2, \dots$ by applying complex exponentials for each of these frequencies and measuring the gain. While this could be feasible in simulation, it may not be done for systems in the physical world, which act on real inputs to produce real outputs. Fortunately, such LTI systems, including those described by linear constant-coefficient ordinary differential equations, have conjugate symmetric frequency responses that enable an alternative approach. When $H(j\omega) = H^*(-j\omega)$ for all ω , the response to a sinusoidal input $x(t) = \cos(\omega t)$ will also be a sinusoid $y(t) = A(\omega) \cos(\omega t + \varphi(\omega))$, where $A(\omega) = |H(j\omega)|$ and $\varphi(\omega) = \angle H(j\omega)$.

Q4.1 If you measure the response $y(t)$ of a real-world LTI system to a sinusoidal input $x(t) = \cos(\omega t)$, explain how you can obtain $A(\omega)$ and $\varphi(\omega)$.

The function `system3` implements a system whose frequency response magnitude you should determine experimentally. Generate a time variable spanning 20 s and repeatedly go through the following steps:

- Generate a sinusoidal signal `x` with a chosen frequency ω
- Obtain the output `y = system3(x,samplerate)` and plot it. Because the function simulates processing of a continuous-time signal but operates on a discrete-time sampled version, it requires the sampling rate as one of its input parameters
- Obtain whatever measurements you need from `y`. Gridlines, zoom and data tips, available in the figure window, as well as the `ginput` function, are useful for this.
- Compute $|H(j\omega)|$ from those measurements

Keep in mind that there might be transient effects in the response at the edges of the time variable because you are applying time-truncated sinusoids to the system. Make sure that you take measurements inside a subinterval that does not contain such transients.

Q4.2 Compute $|H(j\omega)|$ of `system3` for $\omega \in \{1, 10, 50, 100, 200, 1000\}$ rad/s using the approach outlined above. Sketch $|H(j\omega)|$ and comment on the type of filtering (lowpass, highpass, bandpass) and whether the filtering response is ideal or not.

Alternative methods of obtaining the frequency response may first measure the impulse response and then compute its Fourier transform. This may be done by directly applying an impulse at the input, which is perfectly viable in simulation, but hardly feasible in practice as it might damage the system. More realistically, the impulse response could be evaluated (and often is) by differentiating the response to a unit step or by computing the crosscorrelation between the input and output to a white noise stimulus.

Q4.3 Apply the approximate impulse `deltaD` to the input of `system3` to obtain (a good approximation of) the impulse response. This requires the sampling step (`1/samplerate`) to be at least 10 times smaller than the width of the impulse, otherwise the approximation will be too coarse. For example,

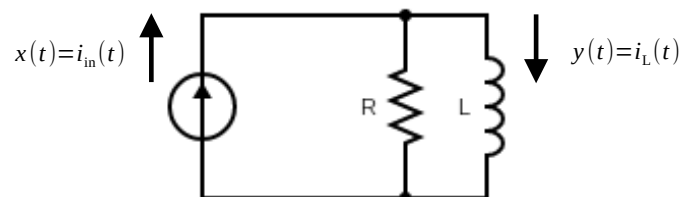
```
t = (-0.2:1/samplerate:1)';  
x = deltaD(t,10/samplerate);  
y = system3(x,samplerate);
```

Compute its Fourier transform as outlined above, zoom into the region $\omega \in [-0, 1000]$ rad/s, and confirm the previously obtained values of $|H(j\omega)|$. Also compute the time constant of this (first-order) system in *two different ways* directly from the impulse response.

Recheck your impulse response in an alternative way by taking the approximate derivative of the response to the unit step

```
ys = system3(u(t),samplerate);  
% Append 0 to "fix" length because diff knocks off one element  
y = [diff(ys); 0]*samplerate;
```

The given function `system3` implements the input-output mapping for the circuit depicted in the figure, where $x(t)$ and $y(t)$ denote the input and output currents, respectively.



Q4.4 Obtain the frequency response of this circuit, $H(j\omega)$, as a function of R and L . You can do this by writing the differential equation, specifying the input and output as $x(t) = e^{j\omega t}$, $y(t) = H(j\omega)e^{j\omega t}$, and then solving for $H(j\omega)$. Alternatively, directly apply the Fourier transform to the differential equation, and use its known properties to factor $X(j\omega)$ and $Y(j\omega)$, such that $H(j\omega) = Y(j\omega)/X(j\omega)$.

Q4.5 For $R = 3\ \Omega$ determine the value of L based on your experimental results. How did you check that this agrees with the measurements of both $H(j\omega)$ and $h(t)$?

5 Filtering and Sampling I

The following functions implement different types of filtering:

- `system4` is a lowpass filter with cutoff frequency of 1 kHz
- `system5` is a highpass filter with cutoff frequency of 800 Hz
- `system6` is a bandpass filter with passband from 600 Hz to 1.2 kHz

```
system4 = @(x) lowpass(x,1e3,samplerate,Steepness=0.99);  
system5 = @(x) highpass(x,8e2,samplerate,Steepness=0.88);  
system6 = @(x) bandpass(x,[6e2 1.2e3],samplerate,Steepness=[0.83 0.98]);
```

Q5.1 Similarly to what you did previously, create a linear chirp signal that will be used as an input to the filters

```
t = (0:1/samplerate:2)';  
x = chirp(t,0,2,2e3);
```

Then, filter `x` through the three different filters to obtain `y4`, `y5`, `y6`, plot them and explain the results

```
y4 = system4(x); y5 = system5(x); y6 = system6(x);  
  
subplot(3,1,1); plot(t,y4); grid  
subplot(3,1,2); plot(t,y5); grid  
subplot(3,1,3); plot(t,y6); grid
```

Implement an alternative to bandpass `filter6` as a cascade (series connection) of lowpass/highpass filters.

Q5.2 Load the file containing the signal `xnoisy`

```
load noisy_signal.mat
```

According to the spectral content of `xnoisy`, determine the passband values for a bandpass filter `system7` that can be applied to `xnoisy` to attenuate its noisy content. Define it similarly to `system6`, using values of `Steepness` to attain lower and upper transition bands of 100 Hz

```
system7 = @(x) bandpass(x,[fpassbpl fpassbph],samplerate,Steepness=[sbpl sbph]);
```

Specifically, if `fpassbpl` and `fpassbph` are the lower and upper edges of the passband, and `w` is the width of the desired transition band, the required values for the lower steepness, `sbpl`, and upper steepness, `sbph`, may be computed as follows

```
sbpl = 1 - w/fpassbpl;
sbph = 1 - w/(samplerate/2 - fpassbph);
```

Listen to the original and filtered signals and compare them in both the time and frequency domains (use `cftransform`). Comment on your results. Can you make an informed guess about the shape of the original signal?

To estimate the impulse response of `system7`, say, `h7`, use an approximate impulse as the input to the filter

```
h7 = system7(deltaD(t,10/samplerate));
plot(t,h7)
```

and, for the frequency response,

```
[H7,f] = cftransform(h7,find(t==0),1e3,samplerate);
plot(f,20*log10(abs(H7)))
```

Plot the impulse response and the frequency response. Are they consistent with your design parameters?

Reset `samplerate = 16e3` and proceed to other topics.

Sampling I

Simulate the sampling process according to the following steps:

- Define `samplerate` as a fine temporal scale. "Continuous-time" signals will actually be simulated as discrete-time signals generated at this scale
- Define the "original continuous-time signal" `xc`
- Define a sampling frequency `fs` which is an integer fraction of `samplerate`, i.e., `fs = samplerate/M`, where the integer `M` is often referred to as the decimation factor
- Generate the "sampled signal" `xd` using the `downsample` function
- If you process `xd` to create some `yd` you can perform the steps required to reconstruct a "continuous-time" signal `yc` from it at the fine temporal scale using the `resample` function

The following code simulates sampling of a continuous-time sinusoid with 10 Hz after sampling with a rate of 100 Hz

```
samplerate = 44.1e3;
t = (0:1/samplerate:4)';
xc = cos(2*pi*10*t);
M = 441; % fs = samplerate/M = 100 Hz
xd = downsample(xc,M);
subplot(2,1,1); plot(t,xc)
subplot(2,1,2); stem(xd)
```

Note above how `stem` is commonly used instead of `plot` to represent discrete-time signals. Now reconstruct the "continuous-time" signal

```

yc = resample(xd,M,1); % yd = xd
yc = resize(yc,numel(xc));
plot(t,[xc yc])

```

Q5.3 Plot `xc`, `yc`, and `xc-yc` and comment on the results.

Repeat for decimation factor $M = 3150$, i.e., a sampling frequency of 14 Hz. Note that, while `xc` and `yc` are now quite different, their values coincide on time instants that are integer multiples of the sampling interval. Measure the frequencies of `xc` and `yc`; are they related to the Nyquist frequency in the way that you expected?

6 Sampling II

Now create and visualise, in the time and frequency domains, the following "continuous-time" signal (don't worry about the details of how `xc1` is generated)

```

t = (-2^15:2^15-1)/samplerate;
W = 2*pi*250;

xc1 = imag(hilbert((W/pi)*sinc(W/pi*t)));
[Xc1,f] = cftransform(xc1,find(t == 0),1e4,samplerate,'whole');

subplot(2,1,1); plot(t,xc1); grid on
subplot(2,1,2); plot(f,imag(Xc1)); grid on

```

Q6.1 Downsample `xc1` by $M = 128$, reconstruct as before, and compare with the original signal

```

M = 128;
yc1 = resample(downsample(xc1,M),M,1);
Yc1 = cftransform(yc1,find(t == 0),f,samplerate);

subplot(2,1,1); plot(t,[xc1 yc1]); grid on
subplot(2,1,2); plot(f,imag([Xc1 Yc1])); grid on

```

Why is the real part of both spectra essentially zero? Explain how the spectrum of `yc1` resulted from that of `xc1`, including how their effective maximum frequencies are related.

Q6.2 In previous sections you plotted the Fourier transform of continuous-time signals by using `freqz`, inside the wrapper `cftransform`, on sampled versions (i.e., in discrete time) and dividing the output by `samplerate`. You should now be able to justify this procedure and clarify under which conditions it holds.

Anti-aliasing prefiltering

Start by loading and playing a music clip

```
[xc,samplerate] = audioread('antialias.wav');
soundsc(xc,samplerate)
```

and visualizing its spectrum

```
[Xc,f] = cftransform(xc,1,1e3,samplerate);
plot(f,abs(Xc))
```

Similarly to what you have done above, downsample by a large factor and listen to the result (keep in mind the "hidden" processing done by the computer and sound card as the discrete-time signal is converted to a true continuous-time acoustic signal played by the speakers)

```
M = 8;      % Decimation factor
xd = downsample(xc,M);
soundsc(xd,samplerate/M)
```

We will now preprocess the original music signal with an anti-aliasing filter so that downsampling becomes less damaging

```
N = 100;    % Anti-aliasing filter order
aa = fir1(N,1/M);
yc = filter(aa,1,xc);
% Check original and filtered spectra
[Xc,f] = cftransform(xc,1,1e3,samplerate);
Yc = cftransform(yc,1,f,samplerate);
plot(f,abs([Xc Yc]))
```

Listen to `xc` (again) and `yc`. Now downsample as before and play

```
yd = downsample(yc,M);
soundsc(yd,samplerate/M)
```

Q6.3 Compare and explain the differences in perceived quality of the downsampled music signals with and without anti-alias preprocessing.

Aliasing paradox

Read and play another music clip

```
[xc,samplerate] = audioread('paradox.wav');
soundsc(xc,samplerate)
```

Now downsample it by a factor of 3 (don't use an anti-aliasing filter) and play it again.

```
M = 3;  
xd = downsample(xc,M);  
soundsc(xd,samplerate/M)
```

Q6.4 Aliasing destroys spectral components of signals and normally leads to a degradation in quality. Why, then, does the quality of this particular music clip *improve* after downsampling? Does the same conclusion hold if you downsample by other factors, say, 2 or 4?